

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 ARTIFICIAL INTELLIGENCE Practical

PRACTICAL - 1

Problem :- A student wants to travel from Home to College using the available road network. Each road connects different locations, forming a graph. The objective is to implement the Breadth First Search (BFS) and Iterative Depth First Search (DFS) algorithms to find a route from Home to College. After implementing both algorithms, compare their efficiency and performance based on the path found, time to reach, total distance and travelled.

AIM :-

- A) Implement the Breadth First Search algorithm to solve a given problem.
- B) Implement the Iterative Depth First Search algorithm to solve the same problem.
- C) Compare the performance and efficiency of both algorithms.

Code :-

1. Practical_1 Code.py

```
from collections import deque
import time
graph = graph = {
    "Home": [
        ("Bus Stop", 2, "Left"),
        ("Hospital", 3, "Straight"),
        ("Park", 2, "Right")
    ],
    "Bus Stop": [("Railway", 2, "Straight")],
    "Railway": [("Library", 1, "Straight")],
    "Library": [("College", 2, "Straight")],
    "Hospital": [("Temple", 2, "Straight")],
    "Temple": [("Garden", 2, "Straight")],
    "Garden": [("College", 2, "Straight")],
    "Park": [("Mall", 2, "Straight")],
    "Mall": [("Cinema", 2, "Straight")],
    "Cinema": [("Stadium", 2, "Straight")],
    "Stadium": [("College", 3, "Straight")],
    "College": []
}

def bfs(start, goal):
    queue = deque([(start, [start], 0, [])])
    visited = set()
    start_time = time.perf_counter()
    while queue:
```

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 ARTIFICIAL INTELLIGENCE Practical

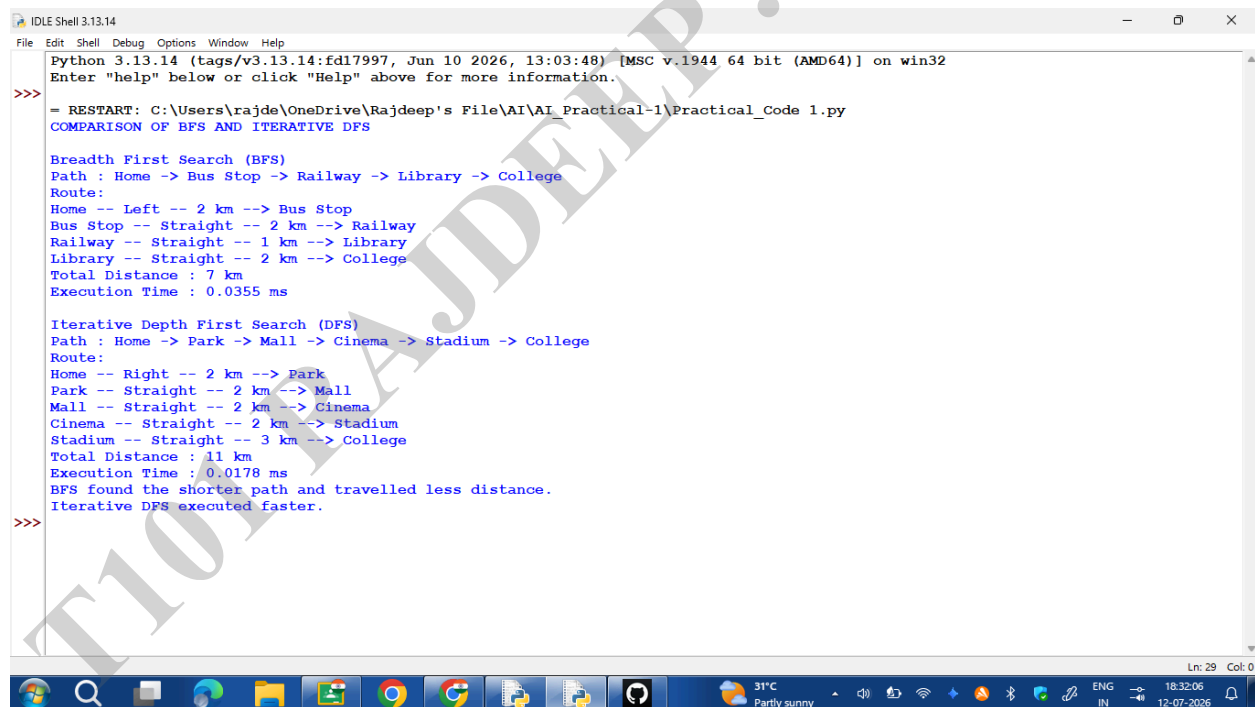
```
node, path, distance, details = queue.popleft()
if node == goal:
    end_time = time.perf_counter()
    return path, distance, details, (end_time - start_time) * 1000
if node not in visited:
    visited.add(node)
    for next_node, dist, direction in graph[node]:
        queue.append((
            next_node,
            path + [next_node],
            distance + dist,
            details + [(node, next_node, dist, direction)]
        ))
def dfs(start, goal):
    stack = [(start, [start], 0, [])]
    visited = set()
    start_time = time.perf_counter()
    while stack:
        node, path, distance, details = stack.pop()
        if node == goal:
            end_time = time.perf_counter()
            return path, distance, details, (end_time - start_time) * 1000
        if node not in visited:
            visited.add(node)
            for next_node, dist, direction in graph[node]:
                stack.append((
                    next_node,
                    path + [next_node],
                    distance + dist,
                    details + [(node, next_node, dist, direction)]
                ))
bfs_path, bfs_distance, bfs_details, bfs_time = bfs("Home", "College")
dfs_path, dfs_distance, dfs_details, dfs_time = dfs("Home", "College")
print("COMPARISON OF BFS AND ITERATIVE DFS")
print("\nBreadth First Search (BFS)")
print("Path :", " -> ".join(bfs_path))
print("Route:")
for source, destination, dist, direction in bfs_details:
    print(source, "--", direction, "--", dist, "km -->", destination)
print("Total Distance :", bfs_distance, "km")
print("Execution Time :", round(bfs_time, 5), "ms")
print("\nIterative Depth First Search (DFS)")
```

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 ARTIFICIAL INTELLIGENCE Practical

```
print("Path :", " -> ".join(dfs_path))
print("Route:")
for source, destination, dist, direction in dfs_details:
    print(source, "--", direction, "--", dist, "km -->", destination)
print("Total Distance :", dfs_distance, "km")
print("Execution Time :", round(dfs_time, 5), "ms")
if bfs_distance < dfs_distance:
    print("BFS found the shorter path and travelled less distance.")
elif dfs_distance < bfs_distance:
    print("DFS found the shorter path.")
else:
    print("Both algorithms travelled the same distance.")
if bfs_time < dfs_time:
    print("BFS executed faster.")
else:
    print("Iterative DFS executed faster.")
```

Output :-



```
IDLE Shell 3.13.14
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
= RESTART: C:\Users\rajde\OneDrive\Rajdeep's File\AI\AI Practical-1\Practical_Code 1.py
COMPARISON OF BFS AND ITERATIVE DFS

Breadth First Search (BFS)
Path : Home -> Bus Stop -> Railway -> Library -> College
Route:
Home -- Left -- 2 km --> Bus Stop
Bus Stop -- Straight -- 2 km --> Railway
Railway -- Straight -- 1 km --> Library
Library -- Straight -- 2 km --> College
Total Distance : 7 km
Execution Time : 0.0355 ms

Iterative Depth First Search (DFS)
Path : Home -> Park -> Mall -> Cinema -> Stadium -> College
Route:
Home -- Right -- 2 km --> Park
Park -- Straight -- 2 km --> Mall
Mall -- Straight -- 2 km --> Cinema
Cinema -- Straight -- 2 km --> Stadium
Stadium -- Straight -- 3 km --> College
Total Distance : 11 km
Execution Time : 0.0178 ms
BFS found the shorter path and travelled less distance.
Iterative DFS executed faster.

>>>
```